

## Contents

|                             |    |
|-----------------------------|----|
| 1 Basic                     | 1  |
| 1.1 default                 | 1  |
| 1.2 奇妙的東西                   | 1  |
| 1.3 random                  | 1  |
| 2 Binary Search             | 1  |
| 2.1 二分搜                     | 1  |
| 2.2 三分搜                     | 1  |
| 3 DataStructure             | 2  |
| 3.1 DSU(並查集)                | 2  |
| 3.2 FenwickTree/BIT         | 2  |
| 3.3 二維 BIT                  | 2  |
| 3.4 Seg/線段樹                 | 2  |
| 3.5 李超線段樹                   | 3  |
| 3.6 SegTree Beats/線段樹 Beats | 3  |
| 4 Graph                     | 4  |
| 4.1 Topo/拓撲                 | 4  |
| 4.2 bellman/邊長鬆弛            | 4  |
| 4.3 ola/歐拉                  | 4  |
| 4.4 LCA/最近公共祖先              | 4  |
| 4.5 Scc/強連通分量               | 5  |
| 4.6 Bcc/雙連通分量               | 5  |
| 5 dynamic programming       | 5  |
| 5.1 SOS DP                  | 5  |
| 5.2 p-median                | 5  |
| 6 Computational Geometry    | 5  |
| 6.1 default                 | 5  |
| 6.2 Circle                  | 6  |
| 6.3 極座標排序                   | 6  |
| 6.4 ConvexHull/凸包           | 6  |
| 6.5 PtSegDist               | 6  |
| 6.6 兩線相交                    | 6  |
| 6.7 點是否在多邊形內                | 6  |
| 6.8 最遠點對/選轉卡尺               | 6  |
| 6.9 最近點對                    | 6  |
| 7 Flow                      | 7  |
| 7.1 km/二分圖最大權匹配             | 7  |
| 7.2 Dinic                   | 7  |
| 7.3 hungarian/匈牙利演算法        | 7  |
| 8 String                    | 8  |
| 8.1 KMP                     | 8  |
| 8.2 Z-value                 | 8  |
| 8.3 馬拉車                     | 8  |
| 8.4 minRotation/最小字典序旋轉     | 8  |
| 8.5 SAIS                    | 8  |
| 8.6 Trie/字典樹                | 9  |
| 8.7 ACautomata/AC 自動機       | 9  |
| 8.8 差分約束系統                  | 9  |
| 9 酷東西可能會用到                  | 9  |
| 9.1 areaofrect/矩形聯集面積       | 9  |
| 10 Math                     | 10 |
| 10.1 basic                  | 10 |
| 10.2 快速幂                    | 10 |
| 10.3 大數運算                   | 10 |
| 10.4 Prime                  | 11 |
| 10.5 ExGCD                  | 11 |
| 10.6 Phi                    | 11 |
| 10.7 Euler/歐拉函數             | 11 |
| 10.8 miller rabin/判斷質數      | 11 |
| 10.9 Pollard Rho/質因數分解      | 11 |
| 10.10 FFT/傅立葉轉換 (多項式相乘)     | 12 |
| 10.11 josephus/約瑟夫問題        | 12 |
| 10.12 Theorem               | 12 |

## 1 Basic

### 1.1 default

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define idonhavegirlfriend ios::sync_with_stdio(0),cin.tie(0);
#define pb push_back
#define endl "\n"
#define all(v) (v).begin(),(v).end()
void solve(){
}
signed main(){
    idonhavegirlfriend
    int _=1;
    //cin>>;
    while(--){
```

```
        solve();
    }
}
```

### 1.2 奇妙的東西

```
priority_queue<T> //由大到小
priority_queue<T,vector<T>,greater<T>> //由小到大
__builtin_popcountll // 二進位有幾個1(記得這是long long)
__builtin_clzll // 左起第一個1之前0的個數
__builtin_parityll // 1的個數的奇偶性
__builtin_mul_overflow(a,b,&h) // a*b是否溢位,h = a * b
;
__builtin_add_overflow(a,b,&h)
```

### 1.3 random

```
mt19937 mt(chrono::steady_clock::now().time_since_epoch().count());
//mt19937_64 mt() -> return random
int randint(int l, int r){
    uniform_int_distribution<> dis(l, r); return dis(mt);
}
```

## 2 Binary Search

### 2.1 二分搜

```
int bsearch_1(int l, int r)
{
    while (l < r)
    {
        int mid = l + r >> 1;
        if (check(mid)) r = mid;
        else l = mid + 1;
    }
    return l;
}
// .....0000000000
int bsearch_2(int l, int r)
{
    while (l < r)
    {
        int mid = l + r + 1 >> 1;
        if (check(mid)) l = mid;
        else r = mid - 1;
    }
    return l;
}
// 000000000.....
int m = *ranges::partition_point(views::iota(0LL,(int)1e9+9),[&](int a){
    return check(a) > k;
});
//[begin,last)
//111111100000000000
//搜左邊數過來第一個 0
//都是 1 會回傳 last
int partitionpoint(int L,int R,function<bool(int)> chk)
{
    int l = L,r = R-1;
    while(r - l > 10){
        int m = l + (r-l)/2;
        if(chk(m)) l = m;
        else r = m;
    }
    int m = l;
    while(m <= r){
        if(!chk(m)) break;
        ++m;
    }
    if(!chk(m)) return m;
    else return R;
}
//手工
```

### 2.2 三分搜

```
int l = 1, r = 100;
while(l < r) {
    int lmid = l + (r - l) / 3; // l + 1/3區間大小
    int rmid = r - (r - l) / 3; // r - 1/3區間大小
    lans = cal(lmid), rans = cal(rmid);
    // 求凹函數極小值
    if(lans <= rans) r = rmid - 1;
    else l = lmid + 1;
}
```

### 3 DataStructure

#### 3.1 DSU(並查集)

```
struct DSU {
    vector<int> f, sz;
    int cnt;
    DSU(int n) : f(n), sz(n) {
        for (int i = 0; i < n; i++) {
            f[i] = i;
            sz[i] = 1;
        }
    }
    int find(int x) {
        if (x == f[x]) return x;
        return f[x] = find(f[x]);
    }
    void merge(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return;
        if (sz[x] < sz[y])
            swap(x, y);
        f[y] = x;
        sz[x] += sz[y];
        cnt--;
    }
    bool same(int a, int b) {
        return find(a) == find(b);
    }
    int groups() { // 找有幾群
        return cnt;
    }
};
```

#### 3.2 FenwickTree/BIT

```
struct Fenwick { // 這個是1base的
    int n;
    vector<int> s;
    int lowbit(int x) { return x & -x; }
    Fenwick(int _n) {
        n = _n + 1;
        s.clear(), s.resize(n, 0);
    }
    void update(int i, int v) { // 更新數值
        for (; i < n; i += lowbit(i))
            s[i] += v;
    }
    int query(int x) { // 查詢前綴和(有包含)
        int pre = 0;
        for (; x; x -= lowbit(x))
            pre += s[x];
        return pre;
    }
    Fenwick(vector<int> a) { // 建構資料結構
        n = a.size(); // a 是 0-based
        s.clear(), s.resize(n + 1, 0); // s 是 1-based
        for (int i = 1; i <= n; i++)
            update(i, a[i - 1]);
    }
};
// 使用:
Fenwick fw(a); // a is vector
Fenwick fw(n); // n is int
```

#### 3.3 二維 BIT

```
struct BIT {
    int n;
    vector<vector<ll>> bit;
    int lowbit(int x) { return x & -x; }
```

```
BIT(int _n) : n(_n + 1) {
    bit.assign(n, vector<ll>(n, 0));
}
void update(int x, int y, ll val) { // 更新 (x, y)
    for (int i = x; i < n; i += lowbit(i))
        for (int j = y; j < n; j += lowbit(j))
            bit[i][j] += val;
}
ll query(int x, int y) { // 查詢 (1, 1) ~ (x, y) 的和
    ll ans = 0;
    for (int i = x; i; i -= lowbit(i))
        for (int j = y; j; j -= lowbit(j)) ans += bit[i][j];
    return ans;
}
ll range_query(int a, int b, int x, int y) {
    return query(x, y) - query(x, b - 1) -
        query(a - 1, y) + query(a - 1, b - 1);
};
```

#### 3.4 Seg/線段樹

```
struct Seg {
#define cl(x) (x << 1)
#define cr(x) (x << 1) + 1
    int _n;
    vector<int> f, tag;
    vector<int> arr;
    // Seg(int n) : _n(n), f(4 * n), tag(4 * n) {}
    Seg(vector<int> a) {
        _n = a.size();
        f.resize(4 * _n + 1, 0);
        tag.resize(4 * _n + 1, 0);
        arr.resize(_n);
        for (int i = 0; i < _n; i++) {
            arr[i] = a[i];
        }

        build(1, 0, _n - 1);
    }
    Seg(int x) {
        _n = x;
        f.resize(4 * _n + 1, 0);
        tag.resize(4 * _n + 1, 0);
        arr.resize(_n);
    }

    void build(int id, int l, int r) {
        if (l == r) {
            f[id] = arr[l];
            return;
        }
        int mid = (l + r) >> 1;
        build(cl(id), l, mid);
        build(cr(id), mid + 1, r);
        pull(id, l, r);
    }

    void pull(int id, int l, int r) {
        int mid = (l + r) >> 1;
        push(cl(id), l, mid);
        push(cr(id), mid + 1, r);
        f[id] = f[cl(id)] + f[cr(id)];
    }

    void push(int id, int l, int r) {
        if (tag[id]) {
            f[id] += tag[id] * (r - l + 1);
            if (l != r) {
                tag[cl(id)] += tag[id];
                tag[cr(id)] += tag[id];
            }
            tag[id] = 0;
        }
    }

    void update(int id, int l, int r, int x, int v) {
        if (l == r) {
            f[id] = v;
```

```

        return;
    }
    int mid = (l + r) >> 1;
    if (x <= mid) update(cl(id), l, mid, x, v);
    if (mid < x) update(cr(id), mid + 1, r, x, v);
    pull(id, l, r);
}

void update(int id, int l, int r, int ql, int qr,
            int v) {
    push(id, l, r);
    if (ql <= l and qr >= r) {
        tag[id] += v;
        return;
    }
    int mid = (l + r) >> 1;
    if (ql <= mid) {
        update(cl(id), l, mid, ql, qr, v);
    }
    if (qr > mid) {
        update(cr(id), mid + 1, r, ql, qr, v);
    }
    pull(id, l, r);
}

int query(int id, int l, int r, int sl, int sr) {
    push(id, l, r);
    if (sl <= l and sr >= r) {
        return f[id];
    }
    int res = 0;
    int mid = (l + r) >> 1;
    int ll = 0;
    int rr = 0;
    if (sl <= mid) {
        ll = query(cl(id), l, mid, sl, sr);
    }
    if (sr > mid) {
        rr = query(cr(id), mid + 1, r, sl, sr);
    }
    res = ll + rr;
    return res;
}
};

```

### 3.5 李超線段樹

```

//線段樹套上凸包
//我相信我們賽中想得出來
const int inf = numeric_limits<i64>::max()/2;
struct Line {
    // y = ax + b
    i64 a{0}, b{-inf};
    i64 operator()(i64 x) {
        return a * x + b;
    }
};

struct Seg {
    int l, r;
    Seg *ls{}, *rs{};
    Line f{};
    Seg(int l, int r) : l(l), r(r) {}
    void add(Line g) {
        int m = (l+r)/2;
        if (g(m) > f(m)) swap(g, f);
        if (g.b == -inf || r - l == 1) return;
        if (g.a < f.a) {
            if (!ls) ls = new Seg(l, m);
            ls->add(g);
        } else {
            if (!rs) rs = new Seg(m, r);
            rs->add(g);
        }
    }
    i64 qry(i64 x) {
        int m = (l+r) / 2;
        i64 y = f(x);
        if (x < m && ls) y = max({y, ls->qry(x)});
        if (x >= m && rs) y = max({y, rs->qry(x)});
        return y;
    }
};

```

```

};
auto add = [&](Line g, int ql, int qr) { //新增線段 [ql, qr)
}
auto find = [&](auto &&self, Seg * now, int l, int r)
-> void {
    if (ql <= l && r <= qr) {
        now->add(g);
        return;
    }
    int m = (l+r) / 2;
    if (ql < m) {
        if (!now->ls) now->ls = new Seg(l, m);
        self(self, now->ls, l, m);
    }
    if (qr > m) {
        if (!now->rs) now->rs = new Seg(m, r);
        self(self, now->rs, m, r);
    }
}
find(find, st, -ninf, ninf);
};
//Seg *st = new Seg(-ninf, ninf); // [l, r)

```

### 3.6 SegTree Beats/線段樹 Beats

```

//阿傑不知道這是啥
struct SegTreeBeat {
    struct node { int mx, smx, cntmx, sum; };
    #define cl(x) (x<<1)+1
    #define cr(x) (x<<1)+2
    #define mid ((l+r)>>1)
    vector<node> seg;
    vector<int> tag;
    vector<int> *a;
    int n;
    void pull(int id) {
        seg[id].sum = seg[cl(id)].sum + seg[cr(id)].sum;
    }
    if (seg[cl(id)].mx == seg[cr(id)].mx) {
        seg[id].mx = seg[cl(id)].mx;
        seg[id].smx = max(seg[cl(id)].smx, seg[cr(id)].smx);
    }
    seg[id].cntmx = seg[cl(id)].cntmx + seg[cr(id)].cntmx;
} else if (seg[cl(id)].mx > seg[cr(id)].mx) {
    seg[id].mx = seg[cl(id)].mx;
    seg[id].smx = max(seg[cl(id)].smx, seg[cr(id)].mx);
}
seg[id].cntmx = seg[cl(id)].cntmx;
} else {
    seg[id].mx = seg[cr(id)].mx;
    seg[id].smx = max(seg[cr(id)].smx, seg[cl(id)].mx);
}
seg[id].cntmx = seg[cr(id)].cntmx;
}

void init(vector<int> &v) {
    n = v.size();
    seg.resize(n*4+5);
    tag = vector<int>(n*4+5, 0);
    a = &v;
}

void build(int l, int r, int id = 0) {
    if (l == r) {
        seg[id] = { (*a)[l], LLONG_MIN, 1, (*a)[l] };
        return;
    }
    build(l, mid, cl(id));
    build(mid+1, r, cr(id));
    pull(id);
}

void apply_chmin(int id, int v) {
    if (seg[id].mx <= v) return;
    seg[id].sum -= (seg[id].mx - v) * seg[id].cntmx;
    seg[id].mx = v;
}

void apply_add(int id, int l, int r, int v) {
    seg[id].sum += v * (r - l + 1);
    if (seg[id].smx != LLONG_MIN) seg[id].smx += v;
    seg[id].mx += v;
    tag[id] += v;
}
}

```

```

void push(int id,int l,int r){
    if(l==r) return;
    if(tag[id]){
        apply_add(cl(id),l,mid,tag[id]);
        apply_add(cr(id),mid+1,r,tag[id]);
        tag[id]=0;
    }
    apply_chmin(cl(id),seg[id].mx);
    apply_chmin(cr(id),seg[id].mx);
}
void add(int l,int r,int sl,int sr,int v,int id=0){
    if(sl==l && r==sr){
        apply_add(id,l,r,v);
        return;
    }
    push(id,l,r);
    if(sl<=mid) add(l,mid,sl,sr,v,cl(id));
    if(mid<sr) add(mid+1,r,sl,sr,v,cr(id));
    pull(id);
}
void chmin(int l,int r,int sl,int sr,int v,int id=0){
    if(seg[id].mx<=v) return;
    if(sl==l && r==sr && seg[id].smx<v){
        apply_chmin(id,v);
        return;
    }
    push(id,l,r);
    if(sl<=mid) chmin(l,mid,sl,sr,v,cl(id));
    if(mid<sr) chmin(mid+1,r,sl,sr,v,cr(id));
    pull(id);
}
int qry(int l,int r,int sl,int sr,int id=0){
    if(sl==l && r==sr) return seg[id].sum;
    push(id,l,r);
    int ans=0;
    if(sl<=mid) ans+=qry(l,mid,sl,sr,cl(id));
    if(mid<sr) ans+=qry(mid+1,r,sl,sr,cr(id));
    return ans;
}
};

```

## 4 Graph

### 4.1 Topo/拓撲

```

vector<int> edge[MAXN], ans;
int deg[MAXN];

void topo(int n) {
    queue<int> q;

    for (int i = 1; i <= n; i++)
        if (!deg[i])
            q.push(i);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ans.push_back(u);

        for (int v : edge[u]) {
            deg[v]--;
            if (!deg[v])
                q.push(v);
        }
    }
}

```

### 4.2 bellman/邊長鬆弛

```

struct Edge {
    int from, to, weight;
} edge[MAXN];
int dis[MAXN];
void bellman_ford(int n, int m) {
    dis[1] = 0; // 起點的距離一定是 0
    for (int j = 0; j < n - 1; j++) {
        // n 個節點要跑 n - 1 次
        for (int i = 0; i < m; i++) { // 對於所有邊都嘗試鬆弛
            if (dis[edge[i].to] >

```

```

                dis[edge[i].from] + edge[i].weight)
                dis[edge[i].to] =
                    dis[edge[i].from] + edge[i].weight;
            }
        }
    }
}

```

### 4.3 ola/歐拉

```

// 歐拉路徑(無向圖)兩個奇數點或者全偶點
// 歐拉路徑(有向圖)條件:起點out=in+1,終點in=out+1,其他點in=out
// 歐拉迴路(無向圖)所有點偶數點
// 歐拉迴路(有向圖)所有點in=out
vector<int> anse;//邊
vector<int> ansp;//點
vector<bool> vis(m, false);
function<void(int)> dfs=&](int u){
    while(!mp[u].empty()){
        auto [x,e]=mp[u].back();
        mp[u].pop_back();
        if(vis[e]){
            continue;
        }
        vis[e]=true;
        dfs(x);
        anse.pb(e);
    }
    ansp.pb(u);
};

```

### 4.4 LCA/最近公共祖先

```

struct{
    int n, logN;
    vector<vector<int>>> jump;
    vector<int> in, out;
    bool is_ancestor(int x, int y){
        if(x == -1 || y == -1) return 1;
        return in[x] <= in[y] && out[y] <= out[x];
    }
    int query(int x, int y){
        if(is_ancestor(x, y)) return x;
        if(is_ancestor(y, x)) return y;
        for(int i = logN - 1; i >= 0; i--){
            if(!is_ancestor(jump[x][i], y)) x = jump[x][i];
        }
        return jump[x][0];
    }
    void init(int _n, vector<vector<int>>>&v){//size,
        adj array 0base
        n = _n;
        logN = log2(n) + 1;

        jump.resize(n, vector<int>(logN, -1));
        in.resize(n); out.resize(n);

        int timing = 1;
        auto dfs = [&](auto &&self, int x) -> void{
            in[x] = timing++;
            for(auto &i : v[x]){
                jump[i][0] = x;
                self(self, i);
            }
            out[x] = timing++;
            return;
        }; dfs(dfs, 0);

        for(int i = 1; i < logN; i++){
            for(int now = 1; now < n; now++){
                if(jump[now][i-1] == -1) continue;
                jump[now][i] = jump[jump[now][i-1]][i-1];
            }
        }
    }
}LCA;

```

### 4.5 Scc/強連通分量

```
//同一團u,v之間有路徑
//bln:在哪個分輛 E原圖
struct Scc{
    int n, nScc, vst[MXN], bln[MXN];
    vector<int> E[MXN], rE[MXN], vec;
    void init(int _n){
        n = _n;
        for (int i=0; i<= n; i++)
            E[i].clear(), rE[i].clear();
    }
    void addEdge(int u, int v){
        E[u].PB(v); rE[v].PB(u);
    }
    void DFS(int u){
        vst[u]=1;
        for (auto v : E[u]) if (!vst[v]) DFS(v);
        vec.PB(u);
    }
    void rDFS(int u){
        vst[u] = 1; bln[u] = nScc;
        for (auto v : rE[u]) if (!vst[v]) rDFS(v);
    }
    void solve(){
        nScc = 0;
        vec.clear();
        fill(vst, vst+n+1, 0);
        for (int i=0; i<n; i++)
            if (!vst[i]) DFS(i);
        reverse(vec.begin(), vec.end());
        fill(vst, vst+n+1, 0);
        for (auto v : vec)
            if (!vst[v]){
                rDFS(v); nScc++;
            }
    }
};
```

#### 4.6 Bcc/雙連通分量

//雙連通分量定義:每個團塊刪除任意一個點都連通,每個割點都屬於多個雙連通分量  
//割點出現數量>=2,割邊出現數量=1

```
struct BccVertex {
    int n, nScc, step, dfn[MXN], low[MXN];
    vector<int> E[MXN], sccv[MXN];
    int top, stk[MXN];
    void init(int _n) {
        n = _n;
        nScc = step = 0;
        for (int i = 0; i < n; i++) E[i].clear();
    }
    void addEdge(int u, int v) {
        E[u].PB(v);
        E[v].PB(u);
    }
    void DFS(int u, int f) {
        dfn[u] = low[u] = step++;
        stk[top++] = u;
        for (auto v : E[u]) {
            if (v == f) continue;
            if (dfn[v] == -1) {
                DFS(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] >= dfn[u]) {
                    int z;
                    sccv[nScc].clear();
                    do {
                        z = stk[--top];
                        sccv[nScc].PB(z);
                    } while (z != v);
                    sccv[nScc++].PB(u);
                }
            } else
                low[u] = min(low[u], dfn[v]);
        }
    }
    vector<vector<int>> solve() {
        vector<vector<int>> res;
        for (int i = 0; i < n; i++) dfn[i] = low[i] = -1;
        for (int i = 0; i < n; i++)
            if (dfn[i] == -1) {
```

```
                top = 0;
                DFS(i, i);
            }
        REP(i, nScc) res.PB(sccv[i]);
        return res;
    }
} graph;
```

## 5 dynamic programming

### 5.1 SOS DP

```
// 求子集和 或超集和 -> !(mask & (1 << i))
for(int i = 0; i<(1<<N); ++i) F[i] = A[i]; //預處理 狀態權重

for(int i = 0; i < N; ++i)
    for (int s = 0; s < (1<<N); ++s)
        if (s & (1 << i))
            F[s] += F[s ^ (1 << i)];

//窮舉子集合
for(int s = mask; s ; s = (s-1)&mask;)
```

### 5.2 p-median

```
void p_Median(){
    for (int i=1; i<=N; ++i)
        for (int j=i; j<=N; ++j){
            m = (i+j)/2, d[i][j] = 0; // m是中位數, d[i][j]為距離的總和
            for (int k=i; k<=j; ++k) d[i][j] += abs(arr[k] - arr[m]);
        }
    for (int p=1; p<=P; ++p)
        for (int n=1; n<=N; ++n){
            dp[p][n] = 1e9;
            for (int k=p; k<=n; ++k)
                if (dp[p-1][k-1] + d[k][n] < dp[p][n]){
                    dp[p][n] = dp[p-1][k-1] + d[k][n];
                    r[p][n] = k; // 從第k個位置往右到第j個位置
                }
        }
}
```

## 6 Computational Geometry

### 6.1 default

```
int sgn(double x) { return (x > -eps) - (x < eps); }
template<class T>
struct pt{
    T x,y;
    pt(T _x,T _y):x(_x),y(_y){}
    pt():x(0),y(0){}
    pt operator * (T c){ return pt(x * c, y * c); }
    pt operator / (T c){ return pt(x / c, y / c); }
    pt operator + (pt a){ return pt(x + a.x, y + a.y); }
    pt operator - (pt a){ return pt(x - a.x, y - a.y); }
    T operator * (pt a){ return x * a.x + y * a.y; } // 內積
    T operator ^ (pt a){ return x * a.y - y * a.x; } // 外積
    bool operator < (pt a) const { return x < a.x || (x == a.x && y < a.y); } //按照x排
    bool operator==(const pt& a) const { return x == a.x && y == a.y; }
};
template<class T>
ld abs(pt<T> a) { //返回向量長度
    return sqrt((ld)(a.x * a.x + a.y * a.y));
}
template<class T>
struct Line { //直線
    pt<T> a, b;
    pt<T> dir()const{return b-a;}
};
template<class T> //線段
```

```
T ori(pt<T> a, pt<T> b, pt<T> c){ return (b - a) ^ (c - a); }
template<class T> //判斷點是否在線段上
bool PtOnSeg(pt<T> p, Line<T> L) {
    return sgn(ori(L.a, L.b, p)) == 0 and sgn((p - L.a) * (p - L.b)) <= 0;
}
```

## 6.2 Circle

```
struct Cir {
    Pt o;
    ld r;
};
bool disjunct(const Cir &a, const Cir &b) { // 分開 || 外切 -> 1
    return sgn(abs(a.o - b.o) - a.r - b.r) >= 0;
}
bool contain(const Cir &a, const Cir &b) {
    return sgn(a.r - b.r - abs(a.o - b.o)) >= 0;
}
```

## 6.3 極座標排序

```
//由+x軸往逆時針排序
struct point {
    int x, y;
};
long long cross(const point &a, const point &b) {
    return (long long) a.x * b.y - (long long) a.y * b.x;
}
bool cmp(const point &a, const point &b) {
    int ah = (a.y < 0 or (a.y == 0 and a.x < 0));
    int bh = (b.y < 0 or (b.y == 0 and b.x < 0));
    if (ah != bh) return ah < bh;
    return cross(a, b) > 0;
}
void argument_sort(vector<point> &points) {
    sort(points.begin(), points.end(), cmp);
}
```

## 6.4 ConvexHull/凸包

```
template<class T>
vector<pt<T>> Hull(vector<pt<T>> P) {
    sort(all(P));
    P.erase(unique(all(P)), P.end());
    P.insert(P.end(), P.rbegin() + 1, P.rend());
    vector<pt<T>> stk;
    for (auto p : P) {
        auto it = stk.rbegin();
        while (stk.rend() - it >= 2 and \
            ori(*next(it), *it, p) <= 0 and \
            (*next(it) < *it) == (*it < p)) {
            it++;
        }
        stk.resize(stk.rend() - it);
        stk.push_back(p);
    }
    stk.pop_back();
    return stk;
}
```

## 6.5 PtSegDist

```
template<class T> //計算點到線段的距離
ld PtSegDist(pt<T> p, Line<T> l) {
    ld ans = min(abs(p - l.a), abs(p - l.b));
    if (sgn(abs(l.a - l.b)) == 0) return ans;
    if (sgn((l.a - l.b) * (p - l.b)) < 0) return ans;
    // case1
    if (sgn((l.b - l.a) * (p - l.a)) < 0) return ans;
    // case1
    return min(ans, abs(ori(p, l.a, l.b)) / abs(l.a - l.b)); // case2
}
```

## 6.6 兩線相交

```
bool isInter(Line l, Line m) {
    if (PtOnSeg(m.a, l) or PtOnSeg(m.b, l) or
```

```
PtOnSeg(l.a, m) or PtOnSeg(l.b, m))
    return true;
return PtSide(m.a, l) * PtSide(m.b, l) < 0 and
    PtSide(l.a, m) * PtSide(l.b, m) < 0;
}
Pt LineInter(Line l, Line m) { //交點
    double s = ori(m.a, m.b, l.a), t = ori(m.a, m.b, l.b);
    return (l.b * s - l.a * t) / (s - t);
}
```

## 6.7 點是否在多邊形內

```
template<class T> //判斷點是否在多邊形內
int inPoly(pt<T> p, const vector<pt<T>> &P) {
    const int n = P.size();
    int cnt = 0;
    for (int i = 0; i < n; i++) {
        pt<T> a = P[i], b = P[(i + 1) % n];
        if (PtOnSeg(p, {a, b})) return 1; // on edge
        if ((sgn(a.y - p.y) == 1) ^ (sgn(b.y - p.y) == 1))
            cnt += sgn(ori(a, b, p));
    }
    return cnt == 0 ? 0 : 2; // out, in
}
```

## 6.8 最遠點對/選轉卡尺

```
double RotatingCalipers(vector<Pt> &v) { //逆時針
    int n = v.size();
    double dist = 0;
    if (n == 1) return 0;
    if (n == 2) return dis(v[0] - v[1]);
    for (int i = 0, j = 2; i < n; i++) {
        Pt a = v[i], b = v[(i + 1) % n];
        while (cross(v[j], a, b) < cross(v[(j + 1) % n], a, b))
            j = (j + 1) % n;
        dist = max(dist, dis(v[j] - a));
        dist = max(dist, dis(v[j] - b));
    }
    return dist;
}
```

## 6.9 最近點對

```
//最近點對
//O(nlogn)
pair<ld, pair<i32, i32>> ClosestPair(vector<Pt> &P) {
    // ans = dis * dis !!注意ans overflow問題
    if (P.size() == 1) { return {1e200L, {0, 0}}; }
    pair<i32, i32> ansi;
    auto ans = abs2(P[0] - P[1]);
    ansi = {0, 1};
    auto upd = [&](const Pt &a, const Pt &b) {
        auto dis = abs2(a - b);
        if (dis < ans) ans = dis, ansi.FF = a.id, ansi.SS = b.id;
    };
    auto cmpy = [&](const Pt &a, const Pt &b) { return a.y < b.y; };
    vector<Pt> t(P.size() + 1);
    auto rec = [&](auto &&self, i32 l, i32 r) {
        if (r - l <= 3) {
            for (i32 i = l; i <= r; i++)
                for (i32 j = i + 1; j <= r; j++) upd(P[i], P[j]);
            sort(P.begin() + l, P.begin() + r + 1, cmpy);
            return;
        }
        i32 m = (l + r) >> 1;
        auto midx = P[m].x;
        self(self, l, m), self(self, m + 1, r);
        i32 tsz = 0;
        inplace_merge(P.begin() + l, P.begin() + m + 1, P.begin() + r + 1, cmpy);
        for (i32 i = l; i <= r; i++) {
            if (abs(P[i].x - midx) * abs(P[i].x - midx) >= ans) continue;

```

```

        for (i32 j = tsz - 1; j >= 0 && (P[i].y - t[j].y) * (P[i].y - t[j].y) < ans; j--)
            upd(P[i], t[j]);
        t[tsz++] = P[i];
    }
};
sort(all(P));
rec(rec, 0, P.size() - 1);
return make_pair(sqrt(ans), ansi);
}

```

## 7 Flow

### 7.1 km/二分圖最大權匹配

```

struct KM{ // max weight, for min negate the weights
int n, mx[MXN], my[MXN], pa[MXN];
ll g[MXN][MXN], lx[MXN], ly[MXN], sy[MXN];
bool vx[MXN], vy[MXN];
void init(int _n) { // 1-based, N個節點
    n = _n;
    for(int i=1; i<=n; i++) fill(g[i], g[i]+n+1, 0);
}
void addEdge(int x, int y, ll w) {g[x][y] = w;} //左
//邊的集合節點x連邊右邊集合節點y權重為w
void augment(int y) {
    for(int x, z; y; y = z)
        x=pa[y], z=mx[x], my[y]=x, mx[x]=y;
}
void bfs(int st) {
    for(int i=1; i<=n; ++i) sy[i]=INF, vx[i]=vy[i]=0;
    queue<int> q; q.push(st);
    for(;;) {
        while(q.size()) {
            int x=q.front(); q.pop(); vx[x]=1;
            for(int y=1; y<=n; ++y) if(!vy[y]){
                ll t = lx[x]+ly[y]-g[x][y];
                if(t==0){
                    pa[y]=x;
                    if(!my[y]){augment(y);return;}
                    vy[y]=1, q.push(my[y]);
                }else if(sy[y]>t) pa[y]=x, sy[y]=t;
            }
        }
        ll cut = INF;
        for(int y=1; y<=n; ++y)
            if(!vy[y]&&cut>sy[y]) cut=sy[y];
        for(int j=1; j<=n; ++j){
            if(vx[j]) lx[j] -= cut;
            if(vy[j]) ly[j] += cut;
            else sy[j] -= cut;
        }
        for(int y=1; y<=n; ++y) if(!vy[y]&&sy[y]==0){
            if(!my[y]){augment(y);return;}
            vy[y]=1, q.push(my[y]);
        }
    }
}
ll solve(){ // 回傳值為完美匹配下的最大總權重
    fill(mx, mx+n+1, 0); fill(my, my+n+1, 0);
    fill(ly, ly+n+1, 0); fill(lx, lx+n+1, -INF);
    for(int x=1; x<=n; ++x) for(int y=1; y<=n; ++y) //
        1-base
        lx[x] = max(lx[x], g[x][y]);
    for(int x=1; x<=n; ++x) bfs(x);
    ll ans = 0;
    for(int y=1; y<=n; ++y) ans += g[my[y]][y];
    return ans;
} }graph;

```

### 7.2 Dinic

```

struct Dinic{
    struct Edge{ int v,f,re; };
    int n,s,t,level[MXN];
    vector<Edge> E[MXN];
    void init(int _n, int _s, int _t){
        n = _n; s = _s; t = _t;
        for (int i=0; i<n; i++) E[i].clear();
    }
    void add_edge(int u, int v, int f){
        E[u].PB({v,f,SZ(E[v])});
        E[v].PB({u,0,SZ(E[u])-1});
    }
}

```

```

bool BFS(){
    for (int i=0; i<n; i++) level[i] = -1;
    queue<int> que;
    que.push(s);
    level[s] = 0;
    while (!que.empty()){
        int u = que.front(); que.pop();
        for (auto it : E[u]){
            if (it.f > 0 && level[it.v] == -1){
                level[it.v] = level[u]+1;
                que.push(it.v);
            }
        }
    }
    return level[t] != -1;
}
int DFS(int u, int nf){
    if (u == t) return nf;
    int res = 0;
    for (auto &it : E[u]){
        if (it.f > 0 && level[it.v] == level[u]+1){
            int tf = DFS(it.v, min(nf,it.f));
            res += tf; nf -= tf; it.f -= tf;
            E[it.v][it.re].f += tf;
            if (nf == 0) return res;
        }
    }
    if (!res) level[u] = -1;
    return res;
}
int flow(int res=0){
    while (BFS())
        res += DFS(s,2147483647);
    return res;
} }flow;
/*flow.init(n, source, sink); : 初始化點數、源點、匯點
flow.add_edge(u, v, w); : 加入一條點u連到點v容量為w的邊
flow.flow(); : 回傳值為最大流量
*/

```

### 7.3 hungarian/匈牙利演算法

```

//二分圖最大匹配
//左邊有L個點，右邊有R個點
//使用前請先呼叫add_edge()加入邊
//匈牙利演算法 O(VE)
//與km演算法不同，匈牙利演算法只會回傳最大匹配數量，無
//法回傳最大權匹配
struct hungarian {
    int L, R;
    vector<bool> vis;
    vector<int> match;
    vector<vector<int>> E;
    hungarian(int l, int r): //左邊有幾個，右邊有幾個
        L(l), R(r),
        vis(l+r+1),
        match(l+r+1,-1),
        E(l+r+1){}
    void add_edge(int l, int r){//左側第幾個(1-base),
        //右側第幾個(1-base)
        r = L+r;
        E[l].push_back(r);
        E[r].push_back(l);
    }
    bool dfs(int u){
        for(int i : E[u]){
            if(vis[i]) continue; // 有連通且未拜訪
            vis[i] = true; // 紀錄是否走過
            if(match[i] == -1 || dfs(match[i])){
                match[i] = u; match[u] = i; // 紀錄匹配
                return true;
            }
        }
        return false;
    }
    int solve(){
        int ans = 0;
        for(int i = 1; i <= L; i++){
            fill(all(vis),0);
            if(dfs(i)) ans++;
        }
        return ans;
    }
}

```

};

## 8 String

### 8.1 KMP

```
int failure[MXN];
vector<int> KMP(vector<int>& t, vector<int>& p){
    vector<int> ret;
    if (p.size() > t.size()) return {};
    for (int i=1, j=failure[0]=-1; i<p.size(); ++i){
        while (j >= 0 && p[j+1] != p[i])
            j = failure[j];
        if (p[j+1] == p[i]) j++;
        failure[i] = j;
    }
    for (int i=0, j=-1; i<t.size(); ++i){
        while (j >= 0 && p[j+1] != t[i])
            j = failure[j];
        if (p[j+1] == t[i]) j++;
        if (j == p.size()-1){
            ret.push_back( i - p.size() + 1 );
            j = failure[j];
        }
    }
    return ret;
}
/*
vector<int> prefunc(const string& s){
    int n = s.size();
    vector<int> pi(n);
    for(int i=1,j=0;i<n;++i){
        j = pi[i-1];
        while(j && s[j] != s[i]) j = pi[j-1]; //往下找藍框
        if(s[j] == s[i]) ++j; //相等會多一格
        pi[i] = j;
    }
    return pi;
}
*/
```

### 8.2 Z-value

```
int z[MXN];
void Z_value(const string& s) {
    //z[i] = lcp(s[0...],s[i...])
    int i, j, left, right, len = s.size();
    left=right=0; z[0]=len;
    for(i=1;i<len;i++){
        j=max(min(z[i-left],right-i),0LL);
        for(;i+j<len&&s[i+j]==s[j];j++);
        z[i]=j;
        if(i+z[i]>right) {
            right=i+z[i];
            left=i;
        }
    }
}
```

### 8.3 馬拉車

```
//記得s.resize(2*n+1)
//z_value_pal(&s[0],n,a.data());
void z_value_pal(char *s,int len,int *z){
    len=(len<<1)+1;
    for(int i=len-1;i>=0;i--){
        s[i]=i&1?s[i>>1]:'@';
        z[0]=1;
        for(int i=1,l=0,r=0;i<len;i++){
            z[i]=i<r?min(z[l+l-i],r-i):1;
            while(i-z[i]>=0&&i+z[i]<len&&s[i-z[i]]==s[i+z[i]])
                ++z[i];
            if(i+z[i]>r) l=i,r=i+z[i];
        }
    }
}
```

### 8.4 minRotation/最小字典序旋轉

```
//rotate(begin(s),begin(s)+minRotation(s),end(s))
int minRotation(string s) {
    int a = 0, N = s.size(); s += s;
    rep(b,0,N) rep(k,0,N) {
        if(a+k == b || s[a+k] < s[b+k])
            {b += max(0LL, k-1LL); break;}
        if(s[a+k] > s[b+k]) {a = b; break;}
    }
    return a;
}
```

## 8.5 SAIS

```
// 此為後綴陣列模板
// 用法:
// 把要處理的字串轉成整數陣列傳入suffix_array(), 會得到
// SA與H維結果
const int N = 500010; // 有可能要在這就寫原本字串的兩
// 倍長
struct SA{
#define REP(i,n) for (int i=0; i<int(n); i++)
#define REP1(i,a,b) for (int i=(a); i<=int(b); i++)
    bool _t[N*2];
    int _s[N*2], _sa[N*2], _c[N*2], x[N], _p[N], _q[N*2],
        hei[N], r[N];
    int operator [] (int i){ return _sa[i]; }
    void build(int *s, int n, int m){
        memcpy(_s, s, sizeof(int) * n);
        sais(_s, _sa, _p, _q, _t, _c, n, m);
        mkhei(n);
    }
    void mkhei(int n){
        REP(i,n) r[_sa[i]] = i;
        hei[0] = 0;
        REP(i,n) if(r[i]) {
            int ans = i>0 ? max(hei[r[i-1]] - 1, 0) : 0;
            while(_s[i+ans] == _s[_sa[r[i]-1]+ans]) ans++;
            hei[r[i]] = ans;
        }
    }
    void sais(int *s, int *sa, int *p, int *q, bool *t,
        int *c, int n, int z){
        bool uniq = t[n-1] = true, neq;
        int nn = 0, nmzx = -1, *nsa = sa + n, *ns = s + n,
            lst = -1;
#define MS0(x,n) memset((x),0,n*sizeof(*(x)))
#define MAGIC(XD) MS0(sa, n); \
        memcpy(x, c, sizeof(int) * z); \
        XD; \
        memcpy(x + 1, c, sizeof(int) * (z - 1)); \
        REP(i,n) if(sa[i] && !t[sa[i]-1]) sa[x[sa[i]-1]]++ = sa[i]-1; \
        memcpy(x, c, sizeof(int) * z); \
        for(int i = n - 1; i >= 0; i--) if(sa[i] && t[sa[i]-1]) sa[-x[sa[i]-1]] = sa[i]-1;
        MS0(c, z);
        REP(i,n) uniq &= ++c[s[i]] < 2;
        REP(i,z-1) c[i+1] += c[i];
        if (uniq) { REP(i,n) sa[-c[s[i]]] = i; return; }
        for(int i = n - 2; i >= 0; i--) t[i] = (s[i]==s[i+1]) ? t[i+1] : s[i]<s[i+1];
        MAGIC(REP1(i,1,n-1) if(t[i] && !t[i-1]) sa[-x[s[i]]]=p[q[i]=nn++]=i);
        REP(i, n) if (sa[i] && t[sa[i]] && !t[sa[i]-1]) {
            neq=lst<0||memcmp(s+sa[i],s+lst,(p[q[sa[i]]+1]-sa[i])*sizeof(int));
            ns[q[lst=sa[i]]]=nmzx+=neq;
        }
        sais(ns, nsa, p + nn, q + n, t + n, c + z, nn, nmzx + 1);
        MAGIC(for(int i = nn - 1; i >= 0; i--) sa[-x[s[p[nsa[i]]]]] = p[nsa[i]]);
    }
}sa;
int H[ N ], SA[ N ]; // SA: 後綴陣列的length
// H[i]: SA[i]與SA[i -1]的共同子字串
// 長(LCP)
void suffix_array(int* ip, int len) {
    // should padding a zero in the back
    // ip is int array, len is array length
    // ip[0..n-1] != 0, and ip[len] = 0
    ip[len++] = 0;
    sa.build(ip, len, 128);
    for (int i = 0; i < len; i++) {
        H[i] = sa.hei[i + 1];
        SA[i] = sa._sa[i + 1];
    }
    // resulting height, sa array \in [0,len)
}
```

## 8.6 Trie/字典樹

```

struct trie{
    trie *nxt[26];
    int cnt;    //紀錄有多少個字串以此節點結尾
    int sz;    //有多少字串的前綴包括此節點
    trie():cnt(0),sz(0){
        memset(nxt,0,sizeof(nxt));
    }
};

//創建新的字典樹
trie *root = new trie();
// O(|s|)
void insert(string& s){
    trie *now = root;    // 每次從根結點出發
    for(auto i:s){
        now->sz++;
        if(now->nxt[i-'a'] == NULL){
            now->nxt[i-'a'] = new trie();
        }
        now = now->nxt[i-'a'];    //走到下一個字母
    }
    now->cnt++;
    now->sz++;
}

// O(|s|)
int query_prefix(string& s){    //查詢有多少前綴為 s
    trie *now = root;    // 每次從根結點出發
    for(auto i:s){
        if(now->nxt[i-'a'] == NULL){
            return 0;
        }
        now = now->nxt[i-'a'];
    }
    return now->sz;
}

int query_count(string& s){    //查詢字串 s 出現次數
    trie *now = root;    // 每次從根結點出發
    for(auto i:s){
        if(now->nxt[i-'a'] == NULL){
            return 0;
        }
        now = now->nxt[i-'a'];
    }
    return now->cnt;
}

```

## 8.7 ACautomata/AC 自動機

```

//makefail before query
//用於多模式字串匹配
struct ACautomata{
    struct Node{
        int cnt,i;
        Node *go[26], *fail, *dic;
        Node(){
            cnt = 0; fail = 0; dic = 0; i = 0;
            memset(go,0,sizeof(go));
        }
    }pool[1048576],*root;
    int ans[1048576];
    int nMem,n_pattern;
    Node* new_Node(){
        pool[nMem] = Node();
        return &pool[nMem++];
    }
    void init() {
        nMem=0;root=new_Node();n_pattern=0;
        add("");
    }
    void add(const string &str) { insert(root,str,0); }
    void insert(Node *cur, const string &str, int pos){
        for(int i=pos;i<str.size();i++){
            if(!cur->go[str[i]-'a'])
                cur->go[str[i]-'a'] = new_Node();
            cur=cur->go[str[i]-'a'];
        }
        cur->cnt++;
        cur->i=n_pattern++;
    }
    void make_fail(){
        queue<Node*> que;

```

```

        que.push(root);
        while (!que.empty()){
            Node* fr=que.front(); que.pop();
            for (int i=0; i<26; i++){
                if (fr->go[i]){
                    Node *ptr = fr->fail;
                    while (ptr && !ptr->go[i]) ptr = ptr->fail;
                    fr->go[i]->fail=ptr=(ptr?ptr->go[i]:root);
                    fr->go[i]->dic=(ptr->cnt?ptr:ptr->dic);
                    que.push(fr->go[i]);
                }
            }
        }
    }
    void query(string s){
        Node *cur=root;
        for(int i=0;i<(int)s.size();i++){
            while(cur&&!cur->go[s[i]-'a']) cur=cur->fail;
            cur=(cur?cur->go[s[i]-'a']:root);
            if(cur->i>=0) ans[cur->i]++;
            for(Node *tmp=cur->dic;tmp;tmp=tmp->dic)
                ans[tmp->i]++;
        }
    }
}
// ans[i] : number of occurrence of pattern i
}AC;

```

## 8.8 差分約束系統

- 2-SAT :
  - $(a_i \vee a_j) = \text{true} \quad \forall (i, j)$
  - 對於任意限制  $(x \vee y)$
  - 建兩條有向邊 (要多編號  $\neg x$ )
  - $x \rightarrow \neg y$  and  $y \rightarrow \neg x$
  - 跑 scc
  - $\text{scc.blm}[x] < \text{scc.blm}[\neg x] \Leftrightarrow x \text{ is true}$
  - $\text{scc.blm}[\neg x] < \text{scc.blm}[x] \Leftrightarrow x \text{ is false}$
  - $\exists x \text{ which } \text{scc.blm}[x] == \text{scc.blm}[\neg x] \Leftrightarrow \text{無解}$
- 差分約束:
  - $n$  個變數及  $m$  個約束條件
  - 求滿足所有  $x_j - x_i \leq b_k \quad (i, j \in [1, n], k \in [1, m])$
  - 的一組  $x_1 \dots x_n$
  - 可轉成  $x_j - x_i \leq b_k \rightarrow x_j \leq x_i + b_k$
  - 結論就是使得所有  $x_j$  變小以滿足上式
  - 建邊跑 SPFA/Bellman
  - 要多建起點  $s$  連到所有  $i$  且邊權 0,  $\text{dis}[s] = 0$
  - 有負環則無解, 否則起點到所有  $i$  的距離為一組解
  - $x_j - x_i \leq k \Rightarrow \text{addEdge } i \xrightarrow{k} j$
  - $x_j - x_i \geq k \Rightarrow \text{addEdge } j \xrightarrow{-k} i$
  - $x_j = x_i \Rightarrow \text{addEdge } i \xrightarrow{0} j \text{ and } j \xrightarrow{0} i$

## 9 酷東西可能會用到

### 9.1 areaofrect/矩形聯集面積

```

struct AreaofRectangles{
#define cl(x) (x<<1)
#define cr(x) (x<<1|1)
    int n, id, sid;
    pair<int,int> tree[MXN<<3];    // count, area
    vector<int> ind;
    tuple<int,int,int,int> scan[MXN<<1];
    void point(int i, int l, int r){
        if(tree[i].first) tree[i].second = ind[r+1] - ind[l];    //當前區間有被做加值
        else if(l != r){    //沒有就直接拿兩個子節點的第二個數字
            int mid = (l+r)>>1;
            tree[i].second = tree[cl(i)].second + tree[cr(i)].second;
        }
        else tree[i].second = 0;
    }
    void upd(int i, int l, int r, int ql, int qr, int v){
        if(ql <= l && r <= qr){
            tree[i].first += v;
            point(i, l, r); return;
        }
        int mid = (l+r) >> 1;
        if(ql <= mid) upd(cl(i), l, mid, ql, qr, v);
        if(qr > mid) upd(cr(i), mid+1, r, ql, qr, v);
        point(i, l, r);
    }
    void init(int _n){
        n = _n; id = sid = 0;
        ind.clear(); ind.resize(n<<1);

```

```

        fill(tree, tree+(n<<2), make_pair(0, 0));
    }
    void addRectangle(int lx, int ly, int rx, int ry){
        ind[id++] = lx; ind[id++] = rx;
        scan[sid++] = make_tuple(ly, 1, lx, rx);
        scan[sid++] = make_tuple(ry, -1, lx, rx);
    }
    int solve(){
        sort(ind.begin(), ind.end());
        ind.resize(unique(ind.begin(), ind.end()) - ind
            .begin()); // 離散化
        sort(scan, scan + sid);
        int area = 0, pre = get<0>(scan[0]);
        for(int i = 0; i < sid; i++){
            auto [x, v, l, r] = scan[i];
            area += tree[1].second * (x-pre); //
            upd(1, 0, ind.size()-1, lower_bound(ind.
                begin(), ind.end(), l)-ind.begin(),
                lower_bound(ind.begin(), ind.end(), r)-
                    ind.begin()-1, v);
            pre = x;
        }
        return area;
    }
} rect;

```

## 10 Math

### 10.1 basic

[a,b] 有包含  
 (a,b) 沒包含  
 ll mul(ll x,ll y,ll mod){  
 ll ret=x\*y-(ll)((long double)x/mod\*y)\*mod;  
 // LL ret=x\*y-(LL)((long double)x\*y/mod+0.5)\*mod;  
 return ret<0?ret+mod:ret;  
 }  
 ll add(int x,int y,int mod){  
 return (x+y)%mod;  
 }

### 10.2 快速幂

```

int ksm(int x, int y) {
    int ans = 1;
    while (y) {
        if (y & 1)
            ans = ans * x;
        x = x * x;
        y >>= 1;
    }
    return ans;
}

// 矩陣乘法
vector<vector<int>> mul(const vector<vector<int>> &A,
    const vector<vector<int>> &B) {
    if (A.empty() || B.empty() || A[0].empty() || B[0].
        empty()) { return {}; }
    int m = A.size();
    int k = A[0].size();
    int n = B[0].size();
    vector<vector<int>> C(m, vector<int>(n, 0));
    for (int i = 0; i < m; i++) {
        for (int p = 0; p < k; p++) if (A[i][p]) {
            for (int j = 0; j < n; j++) {
                C[i][j] = (C[i][j] + 1LL * A[i][p] * B[p][j])%MOD;
            }
        }
    }
    return C;
}

// 矩陣快速幂
vector<vector<int>> ksm(vector<vector<int>> base, int
    exp) {
    int n = base.size();
    vector<vector<int>> res(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) res[i][i] = 1;

    while (exp > 0) {
        if (exp & 1) res = mul(res, base);
        base = mul(base, base);
    }
}

```

```

        exp >>= 1;
    }
    return res;
}

```

### 10.3 大數運算

```

// 加法
string add(string num1, string num2) {
    string res = "";
    int i = num1.length() - 1, j = num2.length() - 1,
        carry = 0;
    while (i >= 0 || j >= 0 || carry) {
        int sum = carry + (i >= 0 ? num1[i--] - '0' :
            0) + (j >= 0 ? num2[j--] - '0' : 0);
        carry = sum / 10;
        res += to_string(sum % 10);
    }
    reverse(res.begin(), res.end());
    return res;
}

// 減法
string sub(string num1, string num2) {
    string res = "";
    int i = num1.length() - 1, j = num2.length() - 1,
        borrow = 0;
    while (i >= 0) {
        int sub = (num1[i--] - '0') - (j >= 0 ? num2[j--] - '0' : 0) - borrow;
        if (sub < 0) {
            sub += 10;
            borrow = 1;
        } else {
            borrow = 0;
        }
        res += to_string(sub);
    }
    while (res.length() > 1 && res.back() == '0') res.
        pop_back(); // 去除前導0
    reverse(res.begin(), res.end());
    return res;
}

// 乘法
string mul(string num1, string num2) {
    int n = num1.size(), m = num2.size();
    vector<int> res(n + m, 0);

    for (int i = n - 1; i >= 0; i--) {
        for (int j = m - 1; j >= 0; j--) {
            int p = (num1[i] - '0') * (num2[j] - '0');
            int sum = p + res[i + j + 1];

            res[i + j + 1] = sum % 10;
            res[i + j] += sum / 10;
        }
    }
    string ans = "";
    for (int x : res) {
        if (!(ans.empty() && x == 0)) // 跳過前導0
            ans += (char)(x + '0');
    }
    return ans.empty() ? "0" : ans;
}

// 除法
bool smaller(string a, string b) {
    if (a.size() != b.size()) return a.size() < b.size()
        ();
    return a < b;
}

string div(string num1, string num2) {
    string cur = "", ans = "";
    for (char c : num1) {
        cur += c;
        // 去掉前導0
        while (cur.size() > 1 && cur[0] == '0') cur.
            erase(cur.begin());

        int x = 0;
        while (!smaller(cur, num2)) {

```

```

        cur = sub(cur, num2);
        x++;
    }
    ans += (char)(x + '0');
}
// 去掉前導0
while (ans.size() > 1 && ans[0] == '0') ans.erase(
    ans.begin());
return ans;
}
string mod(string num1, string num2) { // 餘數
    string cur = "";

    for (char c : num1) {
        cur += c;
        while (cur.size() > 1 && cur[0] == '0') cur.
            erase(cur.begin());
        while (!smaller(cur, num2)) {
            cur = sub(cur, num2);
        }
    }
    return cur;
}
}

```

## 10.4 Prime

```

/* 12721, 13331, 14341, 75577, 123457, 222557, 556679
* 999983, 1097774749, 1076767633, 100102021, 999997771
* 1001010013, 1000512343, 987654361, 999991231
* 999888733, 98789101, 987777733, 999991921, 1010101333
* 1010102101, 1000000000039, 100000000000037
* 2305843009213693951, 4611686018427387847
* 9223372036854775783, 18446744073709551557 */
int mu[ N ], p_tbl[ N ];
vector<int> primes;
void sieve() {
    mu[ 1 ] = p_tbl[ 1 ] = 1;
    for( int i = 2 ; i < N ; i ++ ){
        if( !p_tbl[ i ] ){
            p_tbl[ i ] = i;
            primes.push_back( i );
            mu[ i ] = -1;
        }
        for( int p : primes ){
            int x = i * p;
            if( x >= M ) break;
            p_tbl[ x ] = p;
            mu[ x ] = -mu[ i ];
            if( i % p == 0 ){
                mu[ x ] = 0;
                break;
            }
        }
    }
}
vector<int> factor( int x ){
    vector<int> fac{ 1 };
    while( x > 1 ){
        int fn = SZ(fac), p = p_tbl[ x ], pos = 0;
        while( x % p == 0 ){
            x /= p;
            for( int i = 0 ; i < fn ; i ++ )
                fac.PB( fac[ pos ++ ] * p );
        }
    }
    return fac;
}
}

```

## 10.5 ExGCD

```

//ax+by=gcd(a,b)
PII gcd(int a, int b){
    if(b == 0) return {1, 0};
    PII q = gcd(b, a % b);
    return {q.second, q.first - q.second * (a / b)};
}

```

## 10.6 Phi

```

//歐拉函數 phi(n) 是指<=n中與n互質的數的個數
int _phi(int n) { // O(sqrtN)
    int res = n, a = n;
    for (int i = 2; i * i <= a; i++) {
        if (a % i == 0) {
            res = res / i * (i - 1);
            while (a % i == 0) a /= i;
        }
    }
}

```

```

    }
}
if (a > 1) res = res / a * (a - 1);
return res;
}

int phi[MXN]; // 建表 最大 1e7
void phi_table(int n) {
    phi[1] = 1;
    for (int i = 2; i <= n; ++i) {
        if (phi[i]) continue;
        for (int j = i; j <= n; j += i) {
            if (phi[j] == 0) phi[j] = j;
            phi[j] = phi[j] / i * (i - 1);
        }
    }
}

```

## 10.7 Euler/歐拉函數

```

int prime[350000]; ULL phi[5000005];
bool v[5000005];
#define N 5000000
void quick_euler() {
    int cnt = 0;
    for(int i = 2; i <= N; ++i) {
        if(!v[i]) prime[++cnt] = i, phi[i] = i - 1; //
            若 i 是質數, φ(i) = i - 1
        for(int j = 1; i * prime[j] <= N && j <= cnt;
            ++j) {
            v[i * prime[j]] = 1; // 不是質數就篩掉
            if(i % prime[j] == 0){
                phi[i * prime[j]] = phi[i] * prime[j];
                break;
            }
            else phi[i * prime[j]] = phi[i] * (prime[j]
                - 1); // 積性函數性質用在這裡
        }
    }
}

```

## 10.8 miller rabin/判斷質數

```

// n < 4,759,123,141      3 : 2, 7, 61
// n < 1,122,004,669,633  4 : 2, 13, 23, 1662803
// n < 3,474,749,660,383  6 : pirms <= 13
// n < 2^64                7 :
// 2, 325, 9375, 28178, 450775, 9780504, 1795265022
// Make sure testing integer is in range [2, n-2] if
// you want to use magic.
ll magic[] = {};
bool witness(ll a, ll n, ll u, int t){
    if(!a) return 0;
    ll x = mypow(a, u, n); // ksm(x, y, mod)
    for(int i = 0; i < t; i++) {
        ll nx = mul(x, x, n);
        if(nx == 1 && x != 1 && x != n-1) return 1;
        x = nx;
    }
    return x != 1;
}
bool miller_rabin(ll n) {
    int s = (7);
    // iterate s times of witness on n
    if(n < 2) return 0;
    if(!(n & 1)) return n == 2;
    ll u = n-1; int t = 0;
    // n-1 = u*2^t
    while(!(u & 1)) u >>= 1, t++;
    while(s--){
        ll a = magic[s] % n;
        if(witness(a, n, u, t)) return 0;
    }
    return 1;
}

```

## 10.9 Pollard Rho/質因數分解

```

//回傳n的其中一個因數
// does not work when n is prime O(n^(1/4))
#define LL __int128

```

```

LL f(LL x, LL c, LL mod){ return add(mul(x,x,mod),c,mod); }
LL pollard_rho(LL n) {
    LL c = 1, x = 0, y = 0, p = 2, q, t = 0;
    while (t++ % 128 or gcd(p, n) == 1) {
        if (x == y) c++, y = f(x = 2, c, n);
        if (q = mul(p, abs(x-y), n)) p = q;
        x = f(x, c, n); y = f(f(y, c, n), c, n);
    }
    return gcd(p, n);
}
/*function<void(int)> fact=&](int x){
    if(x==1) return;
    if(miller_rabin(x)){
        f.push_back(x);return;}
    int tmp=pollard_rho(x);
    fact(tmp);fact(x/tmp);};
*/

```

## 10.10 FFT/傅立葉轉換 (多項式相乘)

```

const int MAXN = 262144<<1;
// (must be 2^k)
// before any usage, run pre_fft() first
typedef long double ld;
typedef complex<ld> cplx; //real() ,imag()
const ld PI = acos(-1);
const cplx I(0, 1);
cplx omega[MAXN+1];
void pre_fft(){
    for(int i=0; i<=MAXN; i++){
        omega[i] = exp(i * 2 * PI / MAXN * I);
    }
}
// n must be 2^k
void fft(int n, cplx a[], bool inv=false){
    int basic = MAXN / n;
    int theta = basic;
    for (int m = n; m >= 2; m >= 1) {
        int mh = m >> 1;
        for (int i = 0; i < mh; i++) {
            cplx w = omega[inv ? MAXN-(i*theta%MAXN) : i*theta%MAXN];
            for (int j = i; j < n; j += m) {
                int k = j + mh;
                cplx x = a[j] - a[k];
                a[j] += a[k];
                a[k] = w * x;
            }
        }
        theta = (theta * 2) % MAXN;
    }
    int i = 0;
    for (int j = 1; j < n - 1; j++) {
        for (int k = n >> 1; k > (i ^ k); k >= 1);
        if (j < i) swap(a[i], a[j]);
    }
    if(inv) for (i = 0; i < n; i++) a[i] /= n;
}
cplx arr[MAXN+1];
inline void mul(int _n, ll a[], int _m, ll b[], ll ans[]){
    int n=1, sum=_n+_m-1;
    while(n<sum)
        n<<=1;
    for(int i=0; i<n; i++){
        double x=(i<_n?a[i]:0), y=(i<_m?b[i]:0);
        arr[i]=complex<double>(x+y, x-y);
    }
    fft(n, arr);
    for(int i=0; i<n; i++){
        arr[i]=arr[i]*arr[i];
        fft(n, arr, true);
        for(int i=0; i<sum; i++){
            ans[i]=(ll)(arr[i].real()/4+0.5);
        }
    }
}

```

## 10.11 josephus/約瑟夫問題

```

int josephus(int n, int m){ //n人每m次
    int ans = 0;
    for (int i=1; i<=n; ++i)
        ans = (ans + m) % i;
    return ans;
}

```

## 10.12 Theorem

- Lucas's Theorem :  
For  $n, m \in \mathbb{Z}^*$  and prime  $P$ ,  $C(m, n) \bmod P = \prod C(m_i, n_i)$  where  $m_i$  is the  $i$ -th digit of  $m$  in base  $P$ .
- Stirling approximation :  
$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n e^{\frac{1}{12n}}$$
- Stirling Numbers(permutation  $|P| = n$  with  $k$  cycles):  
 $S(n, k) = \text{coefficient of } x^k \text{ in } \prod_{i=0}^{n-1} (x+i)$
- Stirling Numbers(Partition  $n$  elements into  $k$  non-empty set):  
$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$
- Pick's Theorem :  $A = i + b/2 - 1$   
 $A$ : Area;  $i$ : grid number in the inner;  $b$ : grid number on the side
- Catalan number :  $C_n = \binom{2n}{n} / (n+1)$   
$$C_n^{n+m} - C_{n+1}^{n+m} = (m+n)! \frac{n-m+1}{n+1} \quad \text{for } n \geq m$$
  
$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$
  
$$C_0 = 1 \quad \text{and} \quad C_{n+1} = 2 \binom{2n+1}{n+2} C_n$$
  
$$C_0 = 1 \quad \text{and} \quad C_{n+1} = \sum_{i=0}^n C_i C_{n-i} \quad \text{for } n \geq 0$$
- Euler Characteristic:  
planar graph:  $V - E + F - C = 1$   
convex polyhedron:  $V - E + F = 2$   
 $V, E, F, C$ : number of vertices, edges, faces(regions), and components
- Kirchhoff's theorem :  
 $A_{ii} = \deg(i), A_{ij} = (i, j) \in E ? -1 : 0$ , Deleting any one row, one column, and cal the  $\det(A)$
- Polya' theorem ( $c$  is number of color,  $m$  is the number of cycle size):  
$$\left( \sum_{i=1}^m c^{\gcd(i, m)} \right) / m$$
- Burnside lemma:  
$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$
- 錯排公式: ( $n$  個人中, 每個人皆不再原來位置的組合數):  
$$dp[0] = 1; dp[1] = 0;$$
  
$$dp[i] = (i-1) * (dp[i-1] + dp[i-2]);$$
- Bell 數 (有  $n$  個人, 把他們拆組的方法總數) :  
$$B_0 = 1$$
  
$$B_n = \sum_{k=0}^n s(n, k) \quad (\text{second - stirling})$$
  
$$B_{n+1} = \sum_{k=0}^n \binom{n}{k} B_k$$
- Wilson's theorem :  
$$(p-1)! \equiv -1 \pmod{p}$$
- Fermat's little theorem :  
$$a^p \equiv a \pmod{p}$$
- Euler's totient function:  
$$A^{B^C} \bmod p = \text{pow}(A, \text{pow}(B, C, p-1)) \bmod p$$
- 歐拉函數降幂公式:  
$$A^B \bmod C = A^{B \bmod \phi(C) + \phi(C)} \bmod C$$
- 6 的倍數:  
$$(a-1)^3 + (a+1)^3 + (-a)^3 + (-a)^3 = 6a$$
- Standard young tableau (標準楊表):  
 $\lambda = (\lambda_1 \geq \dots \geq \lambda_k), \sum \lambda_i = n$  denoted by  $\lambda \vdash n$   
 $\lambda \vdash n$  意思為  $\lambda$  整數拆分  $n$  eg.  $n = 10, \lambda = (6, 4)$  此拆分可表示一種楊表形狀。  
楊表: 第 1 列  $\lambda_1$  行... 第  $k$  列  $\lambda_k$  行的方格圖。  
標準楊表: 每列從左到右遞增; 每行從上到下遞增。  
Let  $T$  為某一 Permutation 跑 RSK 後的標準楊表, 則此 Permutation 的 LDS、LIS 長度分別為  $T$  的列、行數。
- RSK Correspondence:  
A permutation is bijective to  $(P, Q)$  一對標準楊表  
 $P$ : Permutation 跑 RSK 算法的結果, 可為半標準楊表。  
 $Q$ : 可用來還原 Permutation (像排列矩陣)。
- Hook length formula (形狀為  $\lambda$  的標準楊表個數):  
$$f^\lambda = \frac{n!}{\prod h_{\lambda(i, j)}}$$
  
 $h_{\lambda(i, j)}$  = number of pair  $(x, y)$  where  $(x = i \vee y = j) \wedge (x, y) \geq (i, j)$  且  $(x, y)$  落在形狀為  $\lambda$  的表上。  
Recursion:  
(i)  $f^{(0, \dots, 0)} = 1$   
(ii)  $f^{(\lambda_1, \dots, \lambda_m)} = \sum_{k=1}^m f^{(\lambda_1, \dots, \lambda_{k-1}, \lambda_k-1, \lambda_{k+1}, \dots, \lambda_m)}$
- 一個環上相鄰顏色不同的染色數量:  
$$f(n, k) = (k-1)^n + (-1)^n (k-1)$$
  
 $n$  是點的數量,  $k$  是顏色的數量





